

Lab Assignment 1

```
1)n = int(input())
```

```
marks = list(map(int , input().split()))
```

```
def determine_grade(a_p):
```

```
    if a_p > 75:
```

```
        print("Grade: Distinction")
```

```
    elif 60<= a_p <= 75:
```

```
        print("Grade: First Division")
```

```
    elif 50<= a_p < 60:
```

```
        print("Grade: Second Division")
```

```
    elif 40<= a_p < 50:
```

```
        print("Grade: Third Division")
```

```
    else:
```

```
        return "Fail"
```

```
if any(mark < 40 for mark in marks):
```

```
    print("Fail")
```

```
else:
```

```
    total_marks = sum(marks)
```

```
    a_p = (total_marks/(n*100))*100
```

```
print(f"Aggregate Percentage: {a_p:.2f}")
```

```
determine_grade(a_p)
```

```
# print(f"Grade: {determine_grade}")
```

```
2)def fib(n):  
    if (n==1):  
        return 1  
    elif (n<=0):  
        return 0  
    else:  
        return fib(n-1) + fib(n-2)
```

```
n=int(input("Enter terms for Fibonacci series: "))
```

```
for i in range (n):  
    print(fib(i),end=" ")
```

3)#Type Content here...

n = int(input())

i = 1

while(i<=n):

print('* '*i)

i+=1

4)# Type Content here...

```
n = int(input())
```

```
for i in range(1,n+1):
```

```
    for j in range(1, i+1):
```

```
        print(j,end = " ")
```

```
    print()
```

Lab Assignment 2

```
1) arr = list(map(int, input().split()))
```

```
key = int(input())
```

```
for index in range(len(arr)):
```

```
    if arr[index] == key:
```

```
        print(index)
```

```
        break
```

```
else:
```

```
    print("Not found")
```

```
2)height = list(map(int, input().split()))
```

```
max = 0
```

```
for i in range(len(height)):
```

```
    if height[i] > max:
```

```
        max = height[i]
```

```
print(max)
```

Lab Assignment 3

```
1)import numpy as np
```

```
# Input matrices
```

```
print("Enter Matrix A:")
```

```
matrix_a = np.array([list(map(int, input().split())) for i in range(3)])
```

```
print("Enter Matrix B:")
```

```
matrix_b = np.array([list(map(int, input().split())) for i in range(3)])
```

```
s=np.zeros((3,3),dtype=int)
```

```
sub=np.zeros((3,3),dtype=int)
```

```
m=np.zeros((3,3),dtype=int)
```

```
dm=np.zeros((3,3),dtype=int)
```

```
# sub=np.zeros((3,3),dtype=int)
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        s[i][j]=matrix_a[i][j]+matrix_b[i][j]
```


Addition

```
print("Addition (A + B):")
```

```
print(s)
```

Subtraction

```
print("Subtraction (A - B):")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        sub[i][j]=(matrix_a[i][j]-matrix_b[i][j])
```

```
print(sub)
```

Multiplication (element-wise)

```
print("Element-wise Multiplication (A * B):")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        m[i][j]=matrix_a[i][j]*matrix_b[i][j]
```

```
print(m)
```

Matrix multiplication (dot product)

```
print("A dot B:")
```

```
for i in range(3):
```

```
    for j in range (3):
```

```
        for k in range(3):
```

```
            dm[i][j]+=matrix_a[i][k]*matrix_b[k][j]
```

```
print(dm)
```

```
tr=np.zeros((3,3), dtype=int)
```

```
# Transpose
```

```
print("Transpose of A:")
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        tr[i][j]=matrix_a[j][i]
```

```
print(tr)
```

```
2)import numpy as np
```

```
# Input matrices
```

```
print("Enter Array1:")
```

```
arr1 = np.array([list(map(int, input().split())) for i in range(3)])
```

```
print("Enter Array2:")
```

```
arr2 = np.array([list(map(int, input().split())) for i in range(3)])
```

```
# Perform horizontal stacking (hstack)
```

```
# s1=np.hstack((arr1,arr2))
```

```
# s2=np.vstack((arr1,arr2))
```

```
print('Horizontal Stack:')
```

```
print(np.hstack((arr1,arr2)))
```

```
print('Vertical Stack:')
```

```
print(np.vstack((arr1,arr2)))
```

3)import numpy as np

Take user input for the start, stop, and step of the sequence

start = int(input())

stop = int(input())

step = int(input())

Generate the sequence using np.arange()

l=np.array([i for i in range(start,stop,step)])

l=np.arange(start,stop,step)

Print the generated sequence

print(l)

4)import numpy as np

def array_operations(A, B):

Convert A and B to NumPy arrays

A=np.array(A)

B=np.array(B)

Arithmetic Operations

sum_result = A+B

diff_result = A-B

prod_result = A*B

Statistical Operations

mean_A = np.mean(A)

median_A = np.median(A)

std_dev_A = np.std(A)

Bitwise Operations

and_result = A & B

or_result = A | B

xor_result = A ^ B

Output results with one space between each element

```
print("Element-wise Sum:", ' '.join(map(str, sum_result)))
```

```
print("Element-wise Difference:", ' '.join(map(str, diff_result)))
```

```
print("Element-wise Product:", ' '.join(map(str, prod_result)))
```

```
print(f"Mean of A: {mean_A}")
```

```
print(f"Median of A: {median_A}")
```

```
print(f"Standard Deviation of A: {std_dev_A}")
```

```
print("Bitwise AND:", ' '.join(map(str, and_result)))
```

```
print("Bitwise OR:", ' '.join(map(str, or_result)))
```

```
print("Bitwise XOR:", ' '.join(map(str, xor_result)))
```

```
A = list(map(int, input().split())) # Elements of array A
```

```
B = list(map(int, input().split())) # Elements of array B
```

```
array_operations(A, B)
```

5)import numpy as np

inputlist = list(map(int,input().split(" ")))

Original array

original_array = np.array(inputlist)

Create a view

view_array = original_array.view()

Create a copy

copy_array = original_array.copy()

Modify the view

view_array[0] = 99

print("Original array after modifying view:", original_array)

print("View array:", view_array)

Modify the copy

copy_array[1] = 88

print("Original array after modifying copy:", original_array)

print("Copy array:", copy_array)

```
6)import numpy as np
```

```
# Input array from the user
```

```
array1 = np.array(list(map(int, input().split())))
```

```
# Searching
```

```
search_value = int(input("Value to search: "))
```

```
count_value = int(input("Value to count: "))
```

```
broadcast_value = int(input("Value to add: "))
```

```
search_indices = np.where(array1 == search_value)[0]
```

```
print(search_indices)
```

```
# Count occurrences in array1
```

```
count = np.count_nonzero(array1 == count_value)
```

```
print(count)
```

```
# Broadcasting addition
```

```
broadcas_array = array1 + broadcast_value
```

```
print(broadcas_array)
```


Sort the first array

sorted_array = np.sort(array1)

print(sorted_array)

7)import numpy as np

a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)

1. Print all student details

print("All student Details:\n",a)

2. print total students

print("Total Students:", len(a))

3. Print all student Roll numbers

print("All Student Roll Nos", a[:,0])

4. Print subject 1 marks

print("Subject 1 Marks", a[:,1])

5. print minimum marks of Subject 2

print("Min marks in Subject 2", a[:,2].min())

6. print maximum marks of Subject 3

```
print("Max marks in Subject 3", a[:,3].max() )
```

7. Print All subject marks

```
a1 = np.delete(a,0,1)
```

```
print("All subject marks:", a1 )
```

8. print Total marks of students

```
print("Total Marks", a[:,1]+a[:,2]+a[:,3] )
```

9. print average marks of each student

```
a2 = np.array(a[:,1]+a[:,2]+a[:,3])
```

```
A = np.around(a2/3,1)
```

```
print(A)
```

10. print average mass of each subject

```
s1 = (a[:,1]).mean()
```

```
s2 = (a[:,2]).mean()
```

```
s3 = (a[:,3]).mean()
```

```
A1 = np.array((s1,s2,s3))
```

```
print("Average Marks of each subject", A1 )
```

11. print average marks of S1 and S2

A2 = np.array((s1,s2))

print("Average Marks of S1 and S2", A2)

12. print average marks of S1 and S3

A3 = np.array((s1,s3))

print("Average Marks of S1 and S3", A3)

13. print Roll number who got maximum marks in Subject 3

i = (a[:,3]).argmax()

r = a[i][0]

print("Roll no who got maximum marks in Subject 3", r)

14. print Roll number who got minimum marks in Subject 2

I = (a[:,2]).argmin()

R = a[I][0]

print("Roll no who got minimum marks in Subject 2", R)

15. print Roll number who got 24 marks in Subject 2

i1 = np.argwhere(a == 24)

r1 = (a[(a[:,2])== 24 ,0]).reshape(-1,1)

print("Roll no who got 24 marks in Subject 2", r1)

16. print count of students who got marks in Subject 1 < 40

```
count = np.argwhere(a[:,1]<40)
```

```
c = len(count)
```

```
print("Count of students who got marks in Subject 1 < 40", c )
```

17. print count of students who got marks in Subject 2 > 90

```
count1 = np.argwhere(a[:,2]>90)
```

```
c1 = len(count1)
```

```
print("Count of students who got marks in Subject 2 > 90:", c1 )
```

18. print count of students in each subject who got marks >= 90

```
Count =
```

```
[len(np.argwhere(a[:,1]>=90)),len(np.argwhere(a[:,2]>=90)),len(np.argwhere(a[:,3]>=90))]
```

```
Count = np.array(Count)
```

```
print("Count of students in each subject who got marks >= 90:", Count )
```

19. print count of subjects in which each student got marks >= 90

```
print("Roll no:", a[:,0] )
```

```
count0 = []
```

```

# for i in a[:,0]:

#     val = 0

#     index = int(np.argmax(a[:,0]==i))

#     if a[index][1] >= 90:

#         val = val + 1

#     elif a[index][2] >= 90:

#         val = val + 1

#     elif a[index][3] >= 90:

#         val = val +1

#     count0.append(val)

# count0 = np.array(count0)

for i in range(len(a[:,0])):

    val = 0

    if a[i][1]>=90:

        val = val + 1

    elif a[i][2]>=90:

        val = val + 1

    elif a[i][3]>=90:

        val = val + 1

    count0.append(val)

count0 = np.array(count0)

```

```
print("Count of subjects in which student got marks >= 90:", count0 )
```

```
# # 20. Print S1 marks in ascending order
```

```
asc = np.sort(a[:,1])
```

```
print(asc)
```

```
# # 21. Print S1 marks >= 50 and <= 90
```

```
print(a[a[:,1]>=50])
```

```
print(a[a[:,1]<=90])
```

```
# # 22. Print the index position of marks 79
```

```
print(np.where(a[:,1]==79))
```

Lab Assignment 4

```
1)import pandas as pd
```

```
# Prompt the user for the file name
```

```
file_name = input()
```

```
# Load the data
```

```
df = pd.read_csv(file_name)
```

```
# Convert the Date column to datetime format
```

```
df['Date'] = pd.to_datetime(df['Date'])
```

```
# Add a Year-Month column by formatting the Date column
```

```
df['YearMonth'] = df['Date'].dt.to_period('M')
```

```
# Calculate total sales for each row
```

```
df['Total_Sales'] = df['Quantity'] * df['Price']
```

```
# Group the data by Year-Month and calculate total sales for each month
```

```
monthly_sales = df.groupby('YearMonth')['Total_Sales'].sum()
```

```
# Find the month with the highest total sales
```

```
best_month = monthly_sales.idxmax()
```

```
highest_sales = monthly_sales.max()
```

```
print(f"Best month: {best_month}")
```

```
print(f"Total sales: ${highest_sales:.2f}")
```



```
2)import pandas as pd
```

```
# Prompt the user for the file name
```

```
file_name = input()
```

```
# Load the data
```

```
df = pd.read_csv(file_name)
```

```
product_sales = df.groupby('Product')['Quantity'].sum()
```

```
# Find the product with the highest total quantity sold
```

```
best_product = product_sales.idxmax()
```

```
highest_quantity = product_sales.max()
```

```
# Display the result
```

```
print(f"Best selling product: {best_product}")
```

```
print(f"Total quantity sold: {highest_quantity}")
```

```
3)import pandas as pd
```

```
# Prompt the user for the file name
```

```
file_name = input()
```

```
# Load the data
```

```
df = pd.read_csv(file_name)
```

```
# write the code..
```

```
city_sales = df.groupby("City")["Quantity"].sum()
```

```
best_city = city_sales.idxmax()
```

```
# Display the result
```

```
print(f"City sold the most products: {best_city}")
```

```
4)import pandas as pd

from itertools import combinations

from collections import Counter


# Prompt user to input the file name

file_name = input()


# Read data from the specified CSV file

df = pd.read_csv(file_name)


count_pairs=Counter()


# write the code


for date,group in df.groupby("Date"):

    products = group['Product']

    for pair in combinations(sorted(products), 2):

        count_pairs[pair] +=1


# Output the most frequent product pairs


best_count= max(count_pairs.values())

best_pair= [pair for pair,val in count_pairs.items() if val==best_count ]
```

```
for pair in best_pair:
```

```
    print(f"{pair[0]} and {pair[1]}: {best_count} times")
```

```
5)import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
# 1. Display the first 5 rows of the dataset
```

```
print(data.head(5))
```

```
# 2. Display the last 5 rows of the dataset
```

```
print(data.tail(5))
```

```
# 3. Get the shape of the dataset
```

```
print(data.shape)
```

```
# 4. Get a summary of the dataset (info)
```

```
print(data.info())
```

```
# 5. Get basic statistics of the dataset
```

```
print(data.describe())
```

```
# 6. Check for missing values
```

```
print(data.isnull().sum())
```

```
6)import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
data['FamilySize'] = data['SibSp'] + data['Parch']
```

```
df = pd.DataFrame(data)
```

```
# 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
```

```
df['FamilySize'] = df['SibSp'] + df['Parch']
```

```
df['IsAlone'] = (df['FamilySize'] == 0).astype(int)
```

```
# 2. Convert 'Sex' to numeric (male: 0, female: 1)
```

```
df['Sex'] = df['Sex'].map({'male' : 0, 'female' : 1})
```

```
# 3. One-hot encode the 'Embarked' column
```

```
df = pd.get_dummies(df, columns=['Embarked'])
```

```
# 4. Get the mean age of passengers
```

```
mean_age = df['Age'].mean()
```

5. Get the median fare of passengers

```
median_fare = df['Fare'].median()
```

6. Get the number of passengers by class

```
passengers_by_class = df['Pclass'].value_counts()
```

7. Get the number of passengers by gender

```
passengers_by_gender = df['Sex'].value_counts()
```

8. Get the number of passengers by survival status

```
passengers_by_survival = df['Survived'].value_counts()
```

9. Calculate the survival rate

```
survival_rate = df['Survived'].mean()
```

10. Calculate the survival rate by gender

```
survival_rate_by_gender = df.groupby('Sex')['Survived'].mean()
```

```
print(mean_age)
```

```
print(median_fare)
```

```
print(passengers_by_class)
```

```
print(passengers_by_gender)
```

```
print(passengers_by_survival)
```

```
print(survival_rate)
```

```
print(survival_rate_by_gender)
```



```
7)import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
data['FamilySize'] = data['SibSp'] + data['Parch']
```

```
data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
```

```
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

```
df = pd.DataFrame(data)
```

```
# 1. Calculate the survival rate by class
```

```
grouped = df.groupby('Pclass')['Survived'].mean()
```

```
print(grouped)
```

```
# 2. Calculate the survival rate by embarked location
```

```
print(df.groupby('Embarked_S')['Survived'].mean())
```

```
# 3. Calculate the survival rate by family size
```

```
print(df.groupby('FamilySize')['Survived'].mean())
```

```
# 4. Calculate the survival rate by being alone
```

```
print(data.groupby('IsAlone')['Survived'].mean())
```

```
# 5. Get the average fare by class
```

```
print(data.groupby('Pclass')['Fare'].mean())
```

6. Get the average age by class

```
print(data.groupby('Pclass')['Age'].mean())
```

7. Get the average age by survival status

```
print(data.groupby('Survived')['Age'].mean())
```

8. Get the average fare by survival status

```
print(data.groupby('Survived')['Fare'].mean())
```

9. Get the number of survivors by class

```
print(data[data['Survived']==1]['Pclass'].value_counts())
```

10. Get the number of non-survivors by class

```
print(data[data['Survived']==0]['Pclass'].value_counts())
```

8)import pandas as pd

import numpy as np

Load the Titanic dataset

data = pd.read_csv('Titanic-Dataset.csv')

data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

df = pd.DataFrame(data)

1. Get the number of survivors by gender

print(df[df['Survived']==1]['Sex'].value_counts())

print((df.groupby('Sex')['Survived']==1).value_counts())

2. Get the number of non-survivors by gender

print(data[data['Survived']==0]['Sex'].value_counts())

3. Get the number of survivors by embarked location

print(data[data['Survived']==1]['Embarked_S'].value_counts())

4. Get the number of non-survivors by embarked location

print(data[data['Survived']==0]['Embarked_S'].value_counts())

5. Calculate the percentage of children (Age < 18) who survived

print(data[data['Age']<18]['Survived'].mean())

6. Calculate the percentage of adults (Age >= 18) who survived

```
print(data[data['Age']>17]['Survived'].mean())
```

7. Get the median age of survivors

```
print(data[data['Survived']==1]['Age'].median())
```

8. Get the median age of non-survivors

```
print(data[data['Survived']==0]['Age'].median())
```

9. Get the median fare of survivors

```
print(data[data['Survived']==1]['Fare'].median())
```

10. Get the median fare of non-survivors

```
print(data[data['Survived']==0]['Fare'].median())
```

Lab Assignment 5

```
1)import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# Load the Titanic dataset from the CSV file
```

```
df = pd.read_csv('titanic.csv')
```

```
# Set up the figure for 5 subplots
```

```
fig, axes = plt.subplots(3, 2, figsize=(12, 12))
```

```
count=df['Pclass'].value_counts()
```

```
count_gen=df['Gender'].value_counts()
```

```
count_s=df['Survived'].value_counts().sort_index()
```

```
# write the code..
```

```
axes[0,0].bar(count.index,count.values,color=['skyblue'])
```

```
axes[0,0].set_title('Passenger Class Distribution')
```

```
axes[0,0].set_xlabel('Pclass')
```

```
axes[0,0].set_ylabel('Count')
```

```
axes[0,1].pie(count_gen.values,labels=['male','female'],colors=['lightblue','lightcoral'],autopct='%1.1f%%')
```

```
axes [0,1].set_title('Gender Distribution')
```

```
axes[1,0].hist(df['Age'],color=['lightgreen'],edgecolor='black',bins=8)
```

```
axes [1,0].set_title('Age Distribution')
```

```
axes[1,0].set_xlabel('Age')
```

```
axes[1,0].set_ylabel('Frequency')
```

```
axes[1,1].bar(count_s.index,count_s.values,color=['lightblue','lightcoral'])
```

```
axes [1,1].set_title('Survival Count')
```

```
axes[1,1].set_xlabel('Survived (0 = No, 1 = Yes)')
```

```
axes[1,1].set_ylabel('Count')
```

```
axes[2,1].plot()
```

```
axes[2,1].set_xlim(0,1)
```

```
axes[2,1].set_ylim(0,1)
```

```
axes[2,0].scatter(df['Age'],df['Fare'],color=['orange'])
```

```
axes [2,0].set_title('Fare vs Age')
```

```
axes[2,0].set_xlabel('Age')
```

```
axes[2,0].set_ylabel('Fare')
```

```
#plt.subplots_adjust(wspace=7)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
2) import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
# Data Cleaning
```

```
data['Age'].fillna(data['Age'].median(), inplace=True)
```

```
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
```

```
data.drop('Cabin', axis=1, inplace=True)
```

```
# Convert categorical features to numeric
```

```
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
```

```
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

```
# Write your code here for Histogram
```

```
plt.hist(data['Age'].dropna(), bins=30, edgecolor='k')
```

```
plt.xlabel('Age')  
plt.ylabel('Frequency')  
plt.title('Age Distribution')  
plt.show()
```

```
3) import pandas as pd  
import matplotlib.pyplot as plt
```

```
# Load the Titanic dataset  
data = pd.read_csv('Titanic-Dataset.csv')
```

```
# Data Cleaning  
data['Age'].fillna(data['Age'].median(), inplace=True)  
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)  
data.drop('Cabin', axis=1, inplace=True)
```

```
# Convert categorical features to numeric  
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})  
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

```
# Write your code here for Bar Plot for Survival Rate  
survival_counts=data['Survived'].value_counts()
```



```
survival_counts.plot(kind='bar')  
  
plt.title('Survival Count')  
  
plt.xlabel('Survived')  
  
plt.ylabel('Count')  
  
plt.show()
```

4) import pandas as pd

```
import matplotlib.pyplot as plt
```

Load the Titanic dataset

```
data = pd.read_csv('Titanic-Dataset.csv')
```

Data Cleaning

```
data['Age'].fillna(data['Age'].median(), inplace=True)
```

```
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
```

```
data.drop('Cabin', axis=1, inplace=True)
```

Convert categorical features to numeric

```
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
```

```
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

Write your code here for Bar Plot for Survival by Gender

```
grouped=data.groupby('Sex')['Survived'].value_counts().unstack()
```

```
grouped.plot(kind='bar',stacked=True)
```

```
plt.title("Survival by Gender")
```

```
plt.xlabel("Gender")
```

```
plt.ylabel("Count")
```

```
plt.legend(["Not Survived", "Survived"])
```

```
plt.show()
```

```
5) import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
# Data Cleaning
```

```
data['Age'].fillna(data['Age'].median(), inplace=True)
```

```
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
```

```
data.drop('Cabin', axis=1, inplace=True)
```

Convert categorical features to numeric

```
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
```

```
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

Write your code here for Bar Plot for Survival by Pclass

```
grp=data.groupby("Pclass")["Survived"].value_counts().unstack()
```

```
grp.plot(kind="bar",stacked="True")
```

```
plt.ylabel("Count")
```

```
plt.title("Survival by Pclass")
```

```
plt.legend(["Not Survive", "Survived"])
```

```
plt.show()
```

6) import pandas as pd

```
import matplotlib.pyplot as plt
```

Load the Titanic dataset

```
data = pd.read_csv('Titanic-Dataset.csv')
```

Data Cleaning

```
data['Age'].fillna(data['Age'].median(), inplace=True)
```

```
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
```

```
data.drop('Cabin', axis=1, inplace=True)
```

Convert categorical features to numeric

data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

Write your code here for Bar Plot for Survival by Embarked

grp=data.groupby("Embarked_Q")["Survived"].value_counts().unstack()

grp.plot(kind="bar",stacked="True")

plt.legend(["Not Survived","Survived"])

plt.ylabel("Counts")

plt.xlabel("Embarked")

plt.title("Survival by Embarked")

plt.show()

7) import pandas as pd

import matplotlib.pyplot as plt

Load the Titanic dataset

data = pd.read_csv('Titanic-Dataset.csv')

Data Cleaning

```
data['Age'].fillna(data['Age'].median(), inplace=True)

data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)

data.drop('Cabin', axis=1, inplace=True)


# Convert categorical features to numeric

data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)


# Write your code here for Box Plot for Age by Pclass

plt.figure(figsize=(8, 6))

data.boxplot(column='Age', by='Pclass')


# Plot customizations

plt.title('Age by Pclass')

plt.suptitle('') # Remove the default subtitle

plt.xlabel('Pclass')

plt.ylabel('Age')


# Show the plot

plt.show()
```

8) import pandas as pd

import matplotlib.pyplot as plt

Load the Titanic dataset

data = pd.read_csv('Titanic-Dataset.csv')

Data Cleaning

data['Age'].fillna(data['Age'].median(), inplace=True)

data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)

data.drop('Cabin', axis=1, inplace=True)

Convert categorical features to numeric

data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

Write your code here for Box Plot for Age by Survived

plt.figure(figsize=(8, 6))

data.boxplot(column='Age', by='Survived')

Plot customizations

```
plt.title('Age by Survival')  
  
plt.suptitle('') # Remove default subtitle  
  
plt.xlabel('Survived')  
  
plt.ylabel('Age')  
  
  
# Show the plot  
  
plt.show()
```

```
9) import pandas as pd  
  
import matplotlib.pyplot as plt
```

```
# Load the Titanic dataset  
  
data = pd.read_csv('Titanic-Dataset.csv')
```

```
# Data Cleaning  
  
data['Age'].fillna(data['Age'].median(), inplace=True)  
  
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)  
  
data.drop('Cabin', axis=1, inplace=True)
```

```
# Convert categorical features to numeric  
  
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})  
  
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

Write your code here for Box Plot for Fare by Pclass

```
plt.figure(figsize=(8, 6))  
data.boxplot(column='Fare', by='Pclass')  
plt.title('Fare by Pclass')  
plt.suptitle('') # Remove the default subtitle  
plt.xlabel('Pclass')  
plt.ylabel('Fare')  
# plt.grid(False)  
plt.show()
```

10) import pandas as pd

import matplotlib.pyplot as plt

Load the Titanic dataset

data = pd.read_csv('Titanic-Dataset.csv')

Data Cleaning

```
data['Age'].fillna(data['Age'].median(), inplace=True)  
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)  
data.drop('Cabin', axis=1, inplace=True)
```



```
# Convert categorical features to numeric
```

```
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
```

```
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

```
plt.scatter(data['Age'],data['Fare'])
```

```
plt.title("Age vs. Fare")
```

```
plt.xlabel("Age")
```

```
plt.ylabel("Fare")
```

```
plt.show()
```

```
11) import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
# Data Cleaning
```

```
data['Age'].fillna(data['Age'].median(), inplace=True)
```

```
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
```

```
data.drop('Cabin', axis=1, inplace=True)
```

Convert categorical features to numeric

data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

Write your code here for Scatter Plot for Age vs. Fare by Survived

colors= data['Survived'].map({0: '#FF0000', 1: '#0000FF'})

plt.scatter(data['Age'], data['Fare'], color= colors)

plt.title('Age vs. Fare by Survival')

plt.xlabel('Age')

plt.ylabel('Fare')

plt.show()